Guía 1 de 3 — Resumir con groupby y combinar con merge

DPL1046 · Clase 6 · Para repasar en casa, a tu ritmo.

Con una tabla limpia, el siguiente paso es **responder preguntas de negocio**. Para eso necesitas dos herramientas: `groupby` (resumir por grupos) y `merge` (combinar tablas). Esta guía las repasa con Olist.

> Antes de empezar, haz tu rutina de inicio (Guía 1 de la Clase 2) con el `.venv` activado. Los ejemplos usan `import pandas as pd` y `RUTA = "data/raw/"`.

---

1. groupby: separar en montones y calcular

Imagina una pila de **boletas del supermercado**. Quieres saber cuánto gastaste *en cada tipo de producto*. Separas las boletas en montones (frutas, lácteos, aseo) y **sumas cada montón**. Eso es `groupby`: **separar en grupos y calcular algo por grupo**. Es una tabla dinámica de Excel, escrita en código (y se repite con un clic).

La plantilla tiene tres piezas: **grupo**, **columna**, **cálculo**.

```
pagos = pd.read_csv(RUTA + "olist_order_payments_dataset.csv")
clientes = pd.read_csv(RUTA + "olist_customers_dataset.csv")

# Contar filas por grupo
por_estado = clientes.groupby("customer_state").size()
print(por_estado.sort_values(ascending=False).head(5))

# Una suma por grupo
suma = pagos.groupby("payment_type")["payment_value"].sum()
print(suma.round(0))
```

Se lee de izquierda a derecha:

- `clientes.groupby("customer_state").size()` → "de `clientes`, agrupa por estado y **cuenta las filas** de cada grupo".
- `pagos.groupby("payment_type")["payment_value"].sum()` → "agrupa por medio de pago, mira el valor pagado y **súmalo**".
- `sort_values(ascending=False).head(5)` → los 5 mayores (ya lo conoces de la Clase 5).

Varios cálculos a la vez: `.agg`

```
resumen = pagos.groupby("payment_type")["payment_value"].agg(["count", "sum",
"mean"])
print(resumen.sort_values("sum", ascending=False).round(1))
```

`.agg([...])` recibe una **lista de cálculos** y entrega una tabla: una fila por grupo, una columna por cálculo.

Cálculo	Qué entrega
`count` / `size()`	cuántas filas
`sum`	el total
`mean`	el promedio
`min` / `max`	el menor / el mayor

2. merge: unir tablas por una llave

Una base de datos reparte la información en tablas para no repetirla. En una **boleta** dice "cliente 8421"; en la **lista de clientes** dice "cliente 8421 vive en Valparaíso". Para saber cuánto compra cada ciudad, hay que **unir** las dos usando el número de cliente, que aparece en ambas. Esa columna en común es la **llave**.

```
orders = pd.read_csv(RUTA + "olist_orders_dataset.csv")
clientes = pd.read_csv(RUTA + "olist_customers_dataset.csv")
print("orders:", orders.shape, "· clientes:", clientes.shape)

pedidos = orders.merge(clientes, on="customer_id", how="left")
print("Después del merge:", pedidos.shape)       # (99441, 12)
```

- `orders.merge(clientes, ...)` → "a `orders`, únele `clientes`".
- `on="customer_id"` → la **llave** en común.
- `how="left"` → "**conserva todos** los pedidos".
- Resultado: 99.441 filas (las mismas) y **12 columnas** (8 de `orders` + 4 nuevas de `clientes`).

¿left o inner?

how	Qué hace con las filas sin pareja	Cuándo usarlo
`left`	Conserva todas las de la tabla de la izquierda	Para empezar: es lo más seguro
`inner`	Deja solo las que tienen pareja en **ambas**	Cuando solo te interesan las que cruzan

Después de un merge, revisa las filas

El error silencioso más común: **duplicar filas sin darte cuenta**. Por eso siempre se compara el tamaño **antes y después**. Y piensa qué significa cada fila: en `items`, cada fila es un **ítem**, no un pedido.

3. Una pregunta completa: ¿cuánto vendió cada estado?

Necesitamos el **precio** (en `items`), el **estado** (en `clientes`) y el **pedido** que las conecta (en `orders`).

```
orders = pd.read_csv(RUTA + "olist_orders_dataset.csv")
items = pd.read_csv(RUTA + "olist_order_items_dataset.csv")
clientes = pd.read_csv(RUTA + "olist_customers_dataset.csv")

entregados = orders[orders["order_status"] == "delivered"]                # limpiar

tabla = items.merge(entregados, on="order_id", how="inner")               #
combinar
tabla = tabla.merge(clientes, on="customer_id", how="inner")

ventas = tabla.groupby("customer_state")["price"].sum()                   # resumir
ventas = ventas.sort_values(ascending=False).round(0)

print(ventas.head(5))
ventas.to_csv("data/ventas_por_estado.csv")                               # guardar
```

Es la receta de la Clase 5 con **dos pasos nuevos** en el medio: combinar y resumir. Como una cadena de montaje: cada paso entrega su resultado al siguiente. Eso es un **pipeline**.

- Primera unión: a cada ítem le pegamos los datos de su pedido (`order_id`). Con `inner`, solo quedan ítems de pedidos entregados.
- Segunda unión: le pegamos el cliente (`customer_id`), que trae el estado.
- `to_csv` **guarda** la tabla en un archivo: es el camino inverso de `read_csv`.

Errores frecuentes

Qué ves	Por qué pasa	Qué hacer
`KeyError` en el `merge`	La llave no existe con ese nombre en una de las tablas	Compara `columns` de ambas tablas
Las filas se multiplican	La llave se repite en una de las tablas	Compara `shape` antes y después; revisa duplicados
Aparecen `_x` y `_y` en las columnas	Las dos tablas tienen una columna con el mismo nombre	Elige solo las columnas que necesitas antes del merge
El `groupby` imprime algo raro	Falta la columna a calcular	Usa `groupby("grupo")["columna"].sum()`
El resultado cambia al cambiar `how`	Hay filas sin pareja	Es información: pregúntate por qué no cruzan

---

Practica tú

1. ¿Cuántos pedidos hay por `order_status`? (con `groupby`).
2. ¿Cuál es el promedio de `price` en la tabla `items`?
3. ¿Qué pasa con el número de filas si en `02_merge.py` cambias `left` por `inner`?
4. ¿Cómo calculas el **flete total** (`freight_value`) por estado?

▼ **Ver respuestas**

1. `orders.groupby("order_status").size()`
2. `items["price"].mean()` (o `items.groupby(...)` si quieres por grupo).
3. No cambia, porque todos los pedidos tienen cliente. Que no cambie ya dice algo sobre la calidad del dato.
4. Con la `tabla` de arriba: `tabla.groupby("customer_state")["freight_value"].sum()`.

Para llevarte

- `groupby("grupo")["columna"].cálculo()` = separar en montones y calcular.
- `.agg([...])` = varios cálculos a la vez.
- `merge(otra, on="llave", how="left")` = unir por una columna en común.
- **Siempre mira `shape` antes y después de un merge.**
- Un pipeline: leer → limpiar → combinar → resumir → guardar.